

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("Social_Network_Ads.csv")
```

```
df = df.iloc[:,2:]
```

```
df.sample(5)
```

1 to 5 of 5 entries  

index	
	87
	337
	112
	376
	377

Show per page

Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

Train test split

```
from sklearn.model_selection import train_test_split
import pandas as pd

# Re-load the CSV to ensure the full dataframe is available.
# This bypasses the issue where df became empty due to df.iloc[:,2:] in a previous cell.
df_full = pd.read_csv("Social_Network_Ads.csv")

# Extract features (X) and target (y) using column names.
# This assumes 'Age', 'EstimatedSalary', and 'Purchased' are present as column names
# in the Social_Network_Ads.csv, and are the intended features and target.
X = df_full[['Age', 'EstimatedSalary']]
y = df_full['Purchased']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state = 0)
X_train.shape, X_test.shape

((280, 2), (120, 2))
```

Standard scaler

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
# fit the scaler to the train set, it will learn the parameters
scaler.fit(X_train)
# transform train and test sets
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
scaler.mean_
```

```
array([3.78642857e+01, 6.98071429e+04])
```

```
X_train
```

	Age	EstimatedSalary	
92	26	15000	
223	60	102000	
234	38	112000	
232	40	107000	
377	42	53000	
...	...	...	
323	48	30000	
192	29	43000	
117	36	52000	
47	27	54000	
172	26	118000	

280 rows × 2 columns

Next steps:

[Generate code with X_train](#)

[New interactive sheet](#)

X_train_scaled

	Age	EstimatedSalary	
0	-1.163172	-1.584970	
1	2.170181	0.930987	
2	0.013305	1.220177	
3	0.209385	1.075582	
4	0.405465	-0.486047	
...	...	...	
275	0.993704	-1.151185	
276	-0.869053	-0.775237	
277	-0.182774	-0.514966	
278	-1.065133	-0.457127	
279	-1.163172	1.393691	

280 rows × 2 columns

Next steps:

[Generate code with X_train_scaled](#)

[New interactive sheet](#)

```
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_test.columns)
```

by converting X_train_scaled & X_test_scaled in dataframe it shows value in tabular from otherwise it shows in numpy array :

```
np.round(X_train.describe(), 1)
```

	Age	EstimatedSalary	
count	280.0	280.0	
mean	37.9	69807.1	
std	10.2	34641.2	
min	18.0	15000.0	
25%	30.0	43000.0	
50%	37.0	70500.0	
75%	46.0	88000.0	
max	60.0	150000.0	

```
np.round(X_train_scaled.describe(), 1)
```

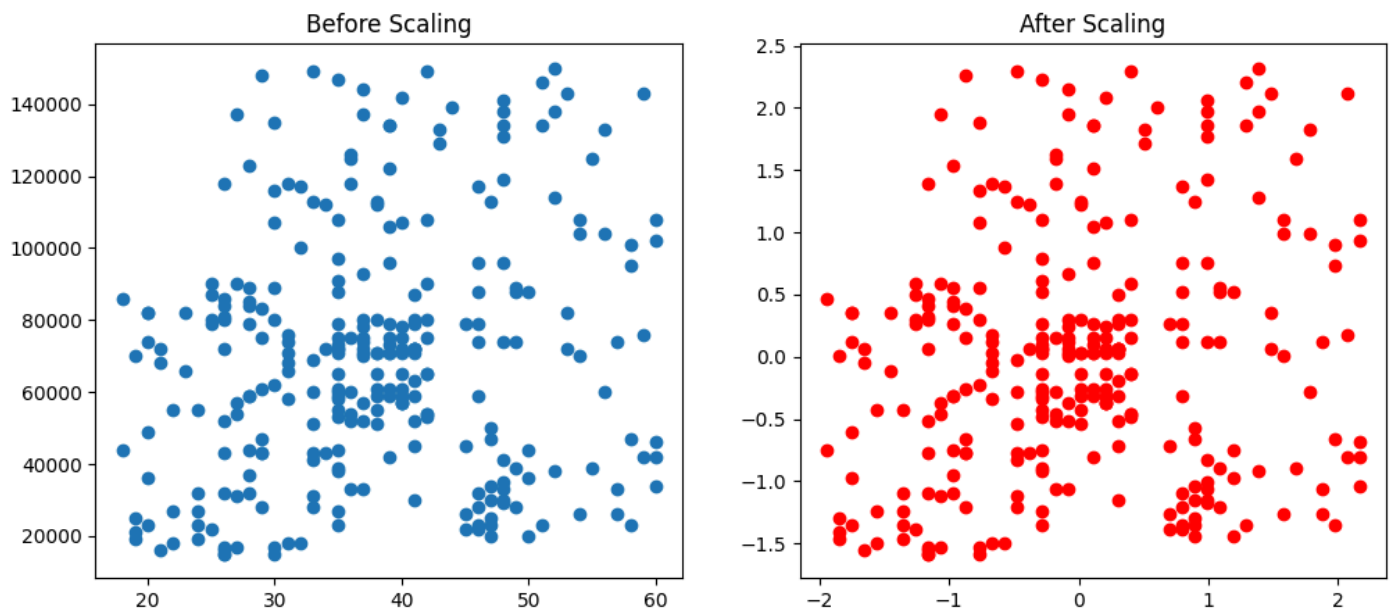
here mean is 0, standard deviation is 1 -> '.'. scaling is done

	Age	EstimatedSalary
count	280.0	280.0
mean	0.0	0.0
std	1.0	1.0
min	-1.9	-1.6
25%	-0.8	-0.8
50%	-0.1	0.0
75%	0.8	0.5
max	2.2	2.3

Effect of scaling

```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12, 5))

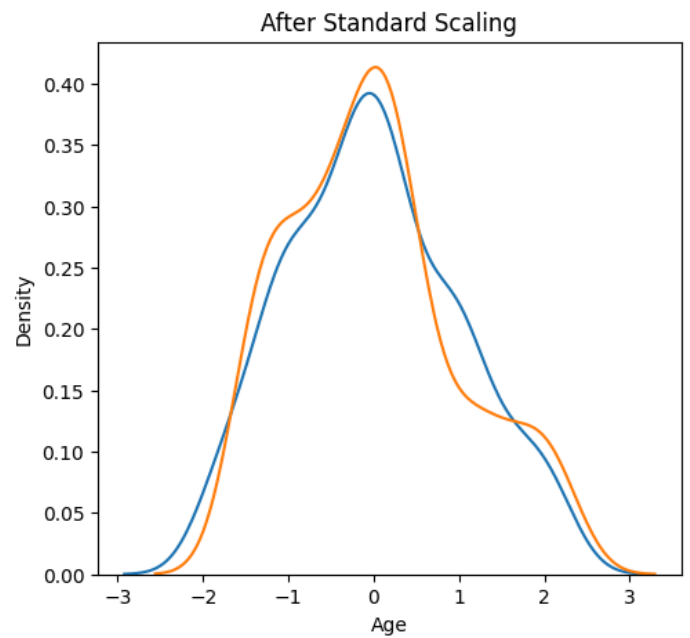
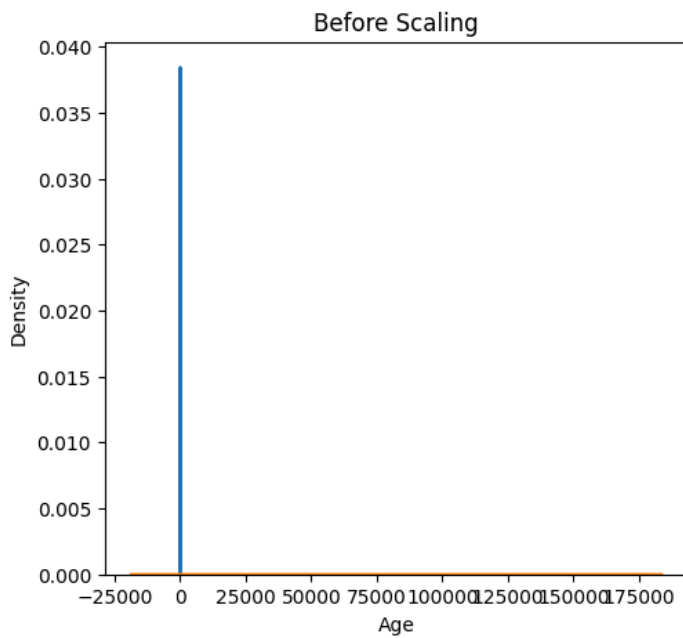
ax1.scatter(X_train['Age'], X_train['EstimatedSalary'])
ax1.set_title('Before Scaling')
ax2.scatter(X_train_scaled['Age'], X_train_scaled['EstimatedSalary'], color='red')
ax2.set_title('After Scaling')
plt.show()
```



```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12, 5))

# before scaling
ax1.set_title('Before Scaling')
sns.kdeplot(X_train['Age'], ax=ax1)
sns.kdeplot(X_train['EstimatedSalary'], ax=ax1)

# after scaling
ax2.set_title('After Standard Scaling')
sns.kdeplot(X_train_scaled['Age'], ax=ax2)
sns.kdeplot(X_train_scaled['EstimatedSalary'], ax=ax2)
plt.show()
# here is probability density function (PDF) plotted
```

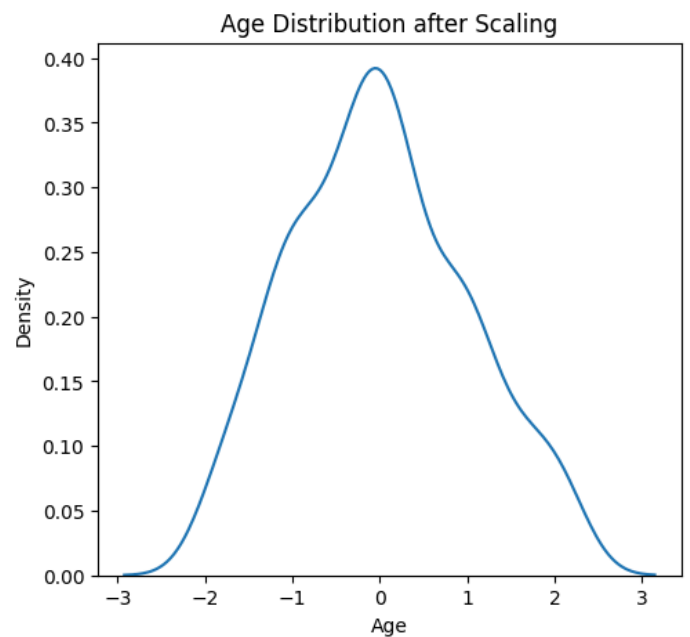
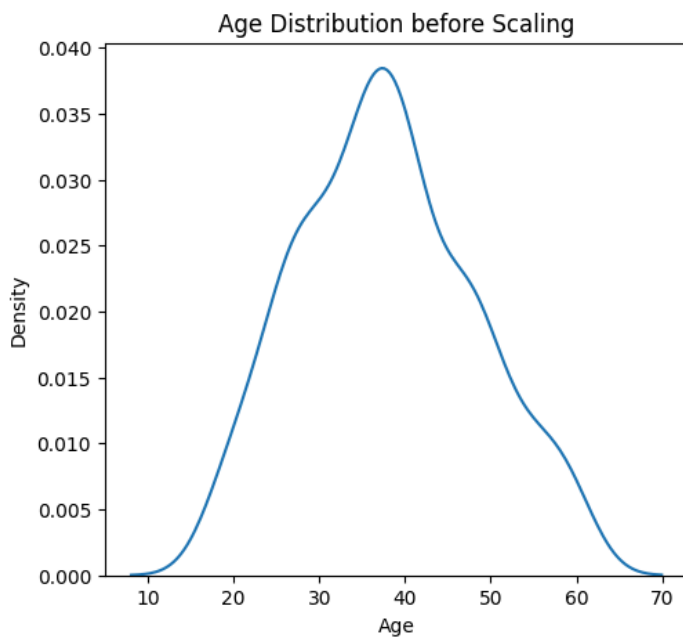


Comparison of Distributions

```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12, 5))

# before scaling
ax1.set_title('Age Distribution before Scaling')
sns.kdeplot(X_train['Age'], ax=ax1)

# after scaling
ax2.set_title('Age Distribution after Scaling')
sns.kdeplot(X_train_scaled['Age'], ax=ax2)
plt.show()
```



Why scaling is important ?

```
from sklearn.linear_model import LogisticRegression
```

```
lr = LogisticRegression()
lr_scaled = LogisticRegression()
```

```
lr.fit(X_train, y_train)
lr_scaled.fit(X_train_scaled, y_train)
```

▼ LogisticRegression ⓘ ?
LogisticRegression()

```
y_pred = lr.predict(X_test)
y_pred_scaled = lr_scaled.predict(X_test_scaled)
```

```
from sklearn.metrics import accuracy_score
```

```
print('Actual', accuracy_score(y_test, y_pred))
print('Scaled', accuracy_score(y_test, y_pred_scaled))
```

If those relationships are already clear enough for Logistic Regression, scaling might not give a big boost to accuracy

```
Actual 0.875
Scaled 0.8666666666666667
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
dt = DecisionTreeClassifier()
dt_scaled = DecisionTreeClassifier()
```

```
dt.fit(X_train, y_train)
dt_scaled.fit(X_train_scaled, y_train)
```

▼ DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier()

```
y_pred = dt.predict(X_test)
y_pred_scaled = dt_scaled.predict(X_test_scaled)
```

```
print('Actual', accuracy_score(y_test, y_pred))
print('Scaled', accuracy_score(y_test, y_pred_scaled))
```

decision tree is an algorithm on which there is no impact of scaling or standardization

```
Actual 0.875
Scaled 0.8666666666666667
```

```
df_full.describe()
```

	Age	EstimatedSalary	Purchased	
count	400.000000	400.000000	400.000000	
mean	37.655000	69742.500000	0.357500	
std	10.482877	34096.960282	0.479864	
min	18.000000	15000.000000	0.000000	
25%	29.750000	43000.000000	0.000000	
50%	37.000000	70000.000000	0.000000	
75%	46.000000	88000.000000	1.000000	
max	60.000000	150000.000000	1.000000	

Effect of Outlier

```
df_full = pd.concat([df_full, pd.DataFrame({'Age':[5,90,95], 'EstimatedSalary':[1000,250000,350000], 'Purchased':[0,1,1]})], ignore_index=True)
```

```
df_full
```

	Age	EstimatedSalary	Purchased	
0	19	19000	0	
1	35	20000	0	
2	26	43000	0	
3	27	57000	0	
4	19	76000	0	
...	...	...	...	
404	90	250000	1	
405	95	350000	1	
406	5	1000	0	
407	90	250000	1	
408	95	350000	1	

409 rows × 3 columns

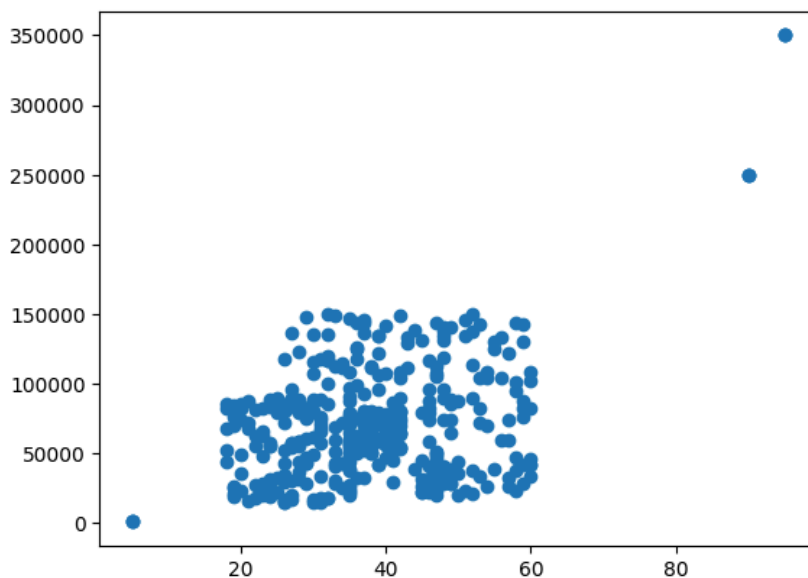
Next steps:

[Generate code with df_full](#)

[New interactive sheet](#)

```
plt.scatter(df_full['Age'], df_full['EstimatedSalary'])
```

<matplotlib.collections.PathCollection at 0x7cc99657a720>



```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(df_full.drop('Purchased', axis=1), df_full['Purchased'], test_size = 0.3,
X_train.shape, X_test.shape
```

```
((286, 2), (123, 2))
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

# fit the scaler to the train set, it will learn the parameters
scaler.fit(X_train)

# transform train and test sets
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_test.columns)
```

```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12, 5))

ax1.scatter(X_train['Age'], X_train['EstimatedSalary'])
ax1.set_title('Before Scaling')
ax2.scatter(X_train_scaled['Age'], X_train_scaled['EstimatedSalary'],color='red')
ax2.set_title('After Scaling')
plt.show()
```

Before Scaling

After Scaling

250000

